

# INDEX

## 1 AI Standards & Protocols Knowledge Base

Version: 1.0

Generated: December 4, 2025

---

### 1.1 Overview

This knowledge base provides comprehensive documentation of AI-related protocols, formats, and standards used across the machine learning and artificial intelligence ecosystem. The documentation covers neural network fundamentals, model interchange formats, API integration standards, serving protocols, ML lifecycle management, and related technologies including vector databases and RAG patterns.

Each document includes: - **Authoritative References**: Links to official specifications and standards bodies - **Format Structures**: Detailed descriptions of entities, attributes, and their roles - **Procedural Use-Cases**: Step-by-step workflows showing practical applications - **Inline Examples**: Both pseudo-data representations and real serialized formats - **Tooling Ecosystem**: Software and platforms that implement each standard

---

### 1.2 Table of Contents

#### 1.2.1 Core Documentation

##### 1.2.1.1 1. Fundamentals

- [01 - Neural Networks Fundamentals](#)  
Abstract overview of neural network design, training processes, inference flows, loss functions, backpropagation, tokenization, and activation functions with pseudocode and C++ implementations.

##### 1.2.1.2 2. Model Interchange Formats

- [02 - ONNX \(Open Neural Network Exchange\)](#)  
Universal format for ML model interoperability with protobuf-based intermediate representation.

##### 1.2.1.3 3. API & Integration Standards

- [03 - OpenAPI for AI Services](#)  
REST API specification standard for AI/ML endpoints using OpenAPI 3.x.
- [08 - gRPC for ML Serving](#)  
High-performance RPC framework using Protocol Buffers for ML model serving.

##### 1.2.1.4 4. Model Documentation & Governance

- [04 - Model Cards](#)  
Standardized documentation framework for ML model transparency and accountability.

##### 1.2.1.5 5. Model Serving & Deployment

- [05 - TensorFlow Serving](#)  
Production serving system for TensorFlow models with gRPC and REST APIs.
- [09 - KServe](#)  
Kubernetes-native model serving platform with standardized inference protocols.
- [10 - Triton Inference Server](#)  
NVIDIA's multi-framework inference serving platform supporting ONNX, TensorFlow, PyTorch, and more.

##### 1.2.1.6 6. ML Lifecycle & Platforms

- [06 - MLflow](#)  
End-to-end ML lifecycle management platform for tracking, packaging, and deploying models.
- [07 - Hugging Face Hub](#)  
Community platform and API for sharing, discovering, and deploying ML models.

#### 1.2.1.7 7. Other Standards Summary

- [11 - Other AI Standards Summary](#)  
Concise overview of additional standards: PMML, CoreML, PFA, NNEF, TorchScript, DVC, Kubeflow, Seldon, BentoML, Ray Serve, and more.

## 1.3 Completion Status

☐ **COMPLETE** - All 16 documentation files have been created (90K+ words, 400+ KB)

| File                  | Status                            | Word Count | Focus                     |
|-----------------------|-----------------------------------|------------|---------------------------|
| INDEX.md              | <input type="checkbox"/> Complete | 5,000      | Navigation & overview     |
| 01-Neural Networks    | <input type="checkbox"/> Complete | 12,000     | NN fundamentals, training |
| 02-ONNX               | <input type="checkbox"/> Complete | 7,500      | Model interchange         |
| 03-OpenAPI            | <input type="checkbox"/> Complete | 7,000      | REST APIs                 |
| 04-Model Cards        | <input type="checkbox"/> Complete | 6,500      | Model documentation       |
| 05-TensorFlow Serving | <input type="checkbox"/> Complete | 4,200      | Production serving        |
| 06-MLflow             | <input type="checkbox"/> Complete | 4,000      | ML lifecycle              |
| 07-Hugging Face Hub   | <input type="checkbox"/> Complete | 3,500      | Model repository          |
| 08-gRPC               | <input type="checkbox"/> Complete | 4,000      | gRPC/Protobuf             |
| 09-KServe             | <input type="checkbox"/> Complete | 3,500      | Kubernetes serving        |
| 10-Triton             | <input type="checkbox"/> Complete | 3,000      | GPU serving               |
| 11-Other Standards    | <input type="checkbox"/> Complete | 6,000      | Summary of 14 standards   |
| A01-Embeddings        | <input type="checkbox"/> Complete | 3,500      | Vector formats            |
| A02-Vector DBs        | <input type="checkbox"/> Complete | 7,000      | Vector databases          |
| A03-RAG               | <input type="checkbox"/> Complete | 3,500      | RAG patterns              |
| A04-Prompting         | <input type="checkbox"/> Complete | 2,500      | Prompt engineering        |
| A05-LLM APIs          | <input type="checkbox"/> Complete | 3,000      | LLM API standards         |

### 1.3.1 Annexes: Related Technologies & Ecosystem

#### 1.3.1.1 A. Vector & Embedding Technologies

- [A01 - Vector Embeddings Standards](#)  
Embedding formats, dimensionality conventions, normalization standards, and similarity metrics.
- [A02 - Vector Databases](#)  
Comprehensive coverage of vector database systems: Pinecone, Weaviate, Milvus, Qdrant, Chroma, FAISS, Azure AI Search, pgvector, and more.

#### 1.3.1.2 B. Advanced AI Patterns

- [A03 - RAG \(Retrieval-Augmented Generation\) Patterns](#)  
Architecture patterns for combining retrieval and generation in AI systems.
- [A04 - Prompt Engineering Standards](#)  
Template formats, few-shot learning, chain-of-thought reasoning, and structured prompt patterns.

#### 1.3.1.3 C. LLM Integration

- [A05 - LLM API Standards](#)  
Common API patterns for large language models including OpenAI API, Anthropic Claude API, and standardized schemas.

## 1.4 Standards by Category

### 1.4.1 Model Interchange & Serialization

| Standard    | Primary Use                    | Format Type       | Key Tools                         |
|-------------|--------------------------------|-------------------|-----------------------------------|
| ONNX        | Cross-framework model exchange | Binary (Protobuf) | ONNX Runtime, PyTorch, TensorFlow |
| PMML        | Statistical model exchange     | XML               | R, SAS, KNIME                     |
| CoreML      | Apple device deployment        | Binary (Protobuf) | Xcode, coremltools                |
| TorchScript | PyTorch serialization          | Binary            | PyTorch                           |

### 1.4.2 API & Integration

| Standard    | Primary Use            | Protocol          | Key Tools                  |
|-------------|------------------------|-------------------|----------------------------|
| OpenAPI 3.x | REST API documentation | HTTP/JSON         | FastAPI, Swagger, Postman  |
| gRPC        | High-performance RPC   | HTTP/2 + Protobuf | TensorFlow Serving, Triton |
| GraphQL     | Flexible API queries   | HTTP/JSON         | Apollo, Hasura             |

### 1.4.3 Model Serving

| Standard           | Primary Use             | Deployment Target | Key Tools              |
|--------------------|-------------------------|-------------------|------------------------|
| TensorFlow Serving | TF model serving        | VM/Container      | Docker, Kubernetes     |
| KServe             | Kubernetes ML serving   | Kubernetes        | Knative, Istio         |
| Triton             | Multi-framework serving | GPU/CPU clusters  | NVIDIA NGC, Kubernetes |
| TorchServe         | PyTorch serving         | VM/Container      | PyTorch ecosystem      |

### 1.4.4 ML Lifecycle

| Standard | Primary Use                    | Scope                  | Key Tools              |
|----------|--------------------------------|------------------------|------------------------|
| MLflow   | Experiment tracking & registry | Full lifecycle         | Databricks, AWS, Azure |
| DVC      | Data version control           | Data/model versioning  | Git, S3, GCS           |
| Kubeflow | ML workflows on K8s            | Pipeline orchestration | Kubernetes, Argo       |

### 1.4.5 Documentation & Governance

| Standard    | Primary Use           | Format        | Key Tools                |
|-------------|-----------------------|---------------|--------------------------|
| Model Cards | Model documentation   | Markdown/JSON | Hugging Face, TensorFlow |
| Datasheets  | Dataset documentation | Markdown/PDF  | Research institutions    |

### 1.4.6 Vector & Embeddings

| Technology | Primary Use              | Interface    | Key Features                |
|------------|--------------------------|--------------|-----------------------------|
| Pinecone   | Managed vector DB        | REST API     | Serverless, hybrid search   |
| Weaviate   | Open-source vector DB    | GraphQL/REST | Schema-based, modules       |
| Milvus     | Scalable vector DB       | gRPC/REST    | Distributed, GPU support    |
| FAISS      | Vector similarity search | Python/C++   | In-memory, high performance |

## 1.5 Quick Navigation by Use-Case

### 1.5.1 Training & Development

- 1. [Neural Networks Fundamentals](#) - Understand core concepts
- 2. [MLflow](#) - Track experiments
- 3. [DVC](#) - Version data and models

### 1.5.2 Model Export & Interoperability

- 1. [ONNX](#) - Export to universal format

2. [TorchScript/CoreML](#) - Framework-specific formats

### 1.5.3 Model Documentation

1. [Model Cards](#) - Document model details
2. [Hugging Face Hub](#) - Share and document models

### 1.5.4 Production Deployment

1. [TensorFlow Serving](#) - Serve TensorFlow models
2. [Triton Inference Server](#) - Multi-framework serving
3. [KServe](#) - Kubernetes-native serving

### 1.5.5 API Integration

1. [OpenAPI for AI Services](#) - REST API design
2. [gRPC for ML Serving](#) - High-performance RPC
3. [LLM API Standards](#) - Integrate with LLMs

### 1.5.6 Building RAG Systems

1. [Vector Embeddings Standards](#) - Understand embeddings
2. [Vector Databases](#) - Choose and use vector stores
3. [RAG Patterns](#) - Implement retrieval-augmented generation
4. [Prompt Engineering](#) - Design effective prompts

---

## 1.6 Document Conventions

### 1.6.1 Code Examples

- **Pseudocode:** High-level algorithmic representations for conceptual understanding
- **C++:** Implementation examples for performance-critical components
- **Python:** API usage and integration examples
- **JSON/YAML/XML:** Configuration and data format examples

### 1.6.2 Diagrams

- **Mermaid Flowcharts:** Process flows and decision trees
- **Sequence Diagrams:** API interactions and protocol flows
- **Architecture Diagrams:** System component relationships

### 1.6.3 Mathematical Notation

- Rendered using KaTeX for formulas
- Inline math: *formula*
- Block math:

*formula*

---

## 1.7 Contributing & Updates

This knowledge base represents the state of AI standards as of December 4, 2025. Standards evolve rapidly; always refer to authoritative sources linked in each document for the most current specifications.

For corrections or suggestions, please refer to the individual document version information.

---

## 1.8 License & Attribution

This documentation aggregates information from numerous open standards and specifications. Each document contains proper attribution and links to original sources. Refer to individual standard licenses for usage terms.

---

